

Decision Process

Qian Wang

School of Biomedical Engineering
ShanghaiTech University

Recap: Probabilistic Models for Sequential Reasoning

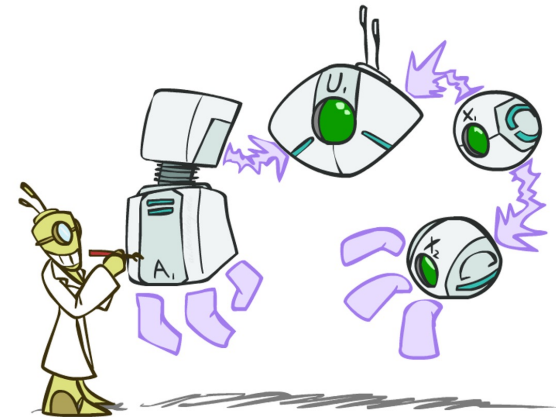
- Bayesian Network (BN):
 - A graphical model representing probabilistic relationships among a set of variables;
 - Used for static modeling, without temporal dynamics;
 - Example: Diagnosing diseases based on symptoms.
- Dynamic Bayesian Network (DBN):
 - An extension of BN that models sequences of variables over time;
 - Captures temporal dependencies between time slices;
 - Example: Modeling weather changes across several days.

Recap: Probabilistic Models for Sequential Reasoning

- Hidden Markov Model (HMM):
 - A special case of DBN with one hidden state variable per time step;
 - Models sequential data with hidden dynamics.
 - Example: Speech recognition
 - Hidden states = phonemes
 - Observations = audio signals

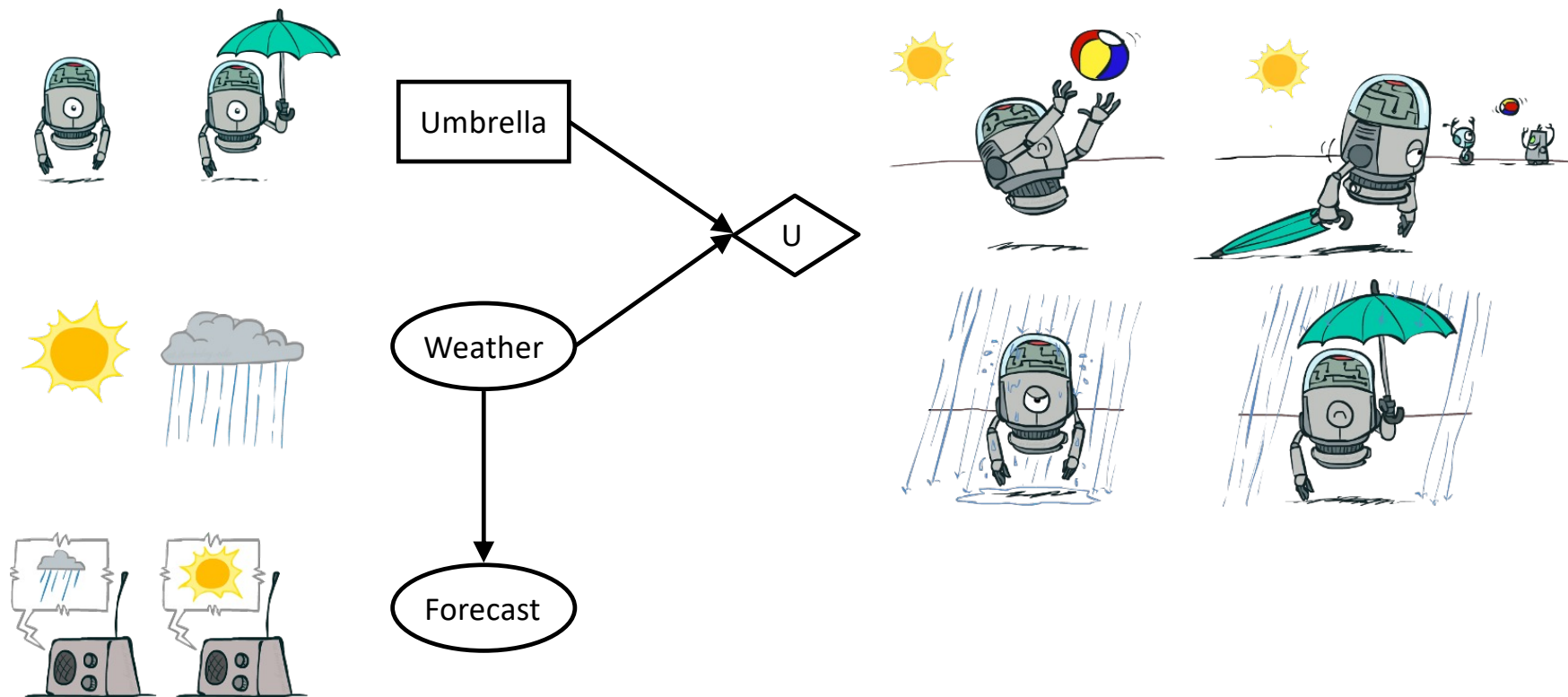
Decision Network

- **Decision network**: A probabilistic graphical model that extends Bayesian Networks by incorporating decision-making and utility.
 - Used to model **rational agents acting** under uncertainty.
 - **State**: The agent's current situation or environment;
 - **Action**: The set of possible choices the agent can make;
 - **Transition**: The resulting state after an action is taken;
 - **Utility**: A numerical measure of how desirable each outcome is.



A **rational** agent should select an **action** that is expected to **maximize** its performance measure quantified by the **utility** function.

Example: Decision Network



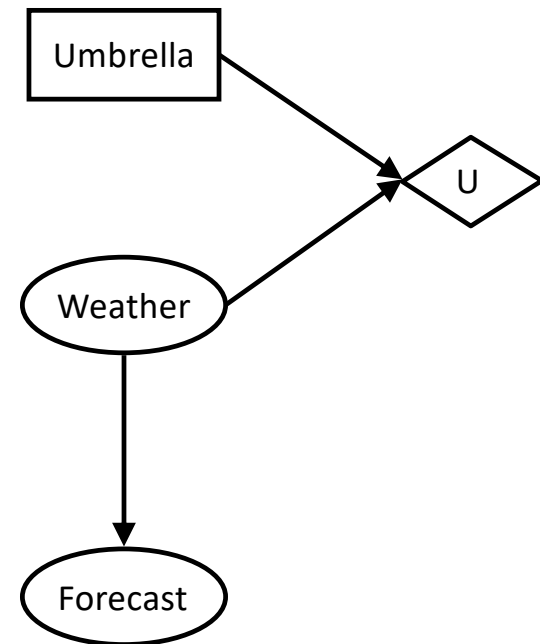
Decision Network

- Decision network combines BN with additional node types for actions and utilities.

○ • **Chance** node (just like BN);

□ • **Action** node (rectangles, cannot have parents, act as observed evidence);

◇ • **Utility** node (diamond, depends on action and chance nodes).



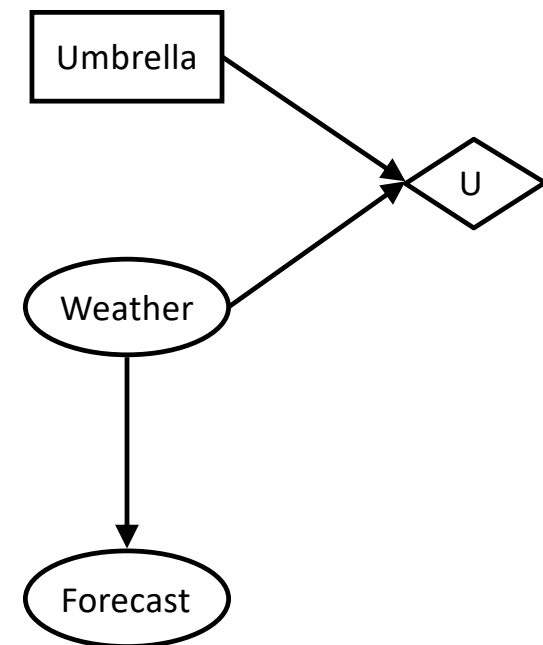
Maximum Expected Utility (MEU)

- **MEU**: choose the action which maximizes the expected utility given the evidence.

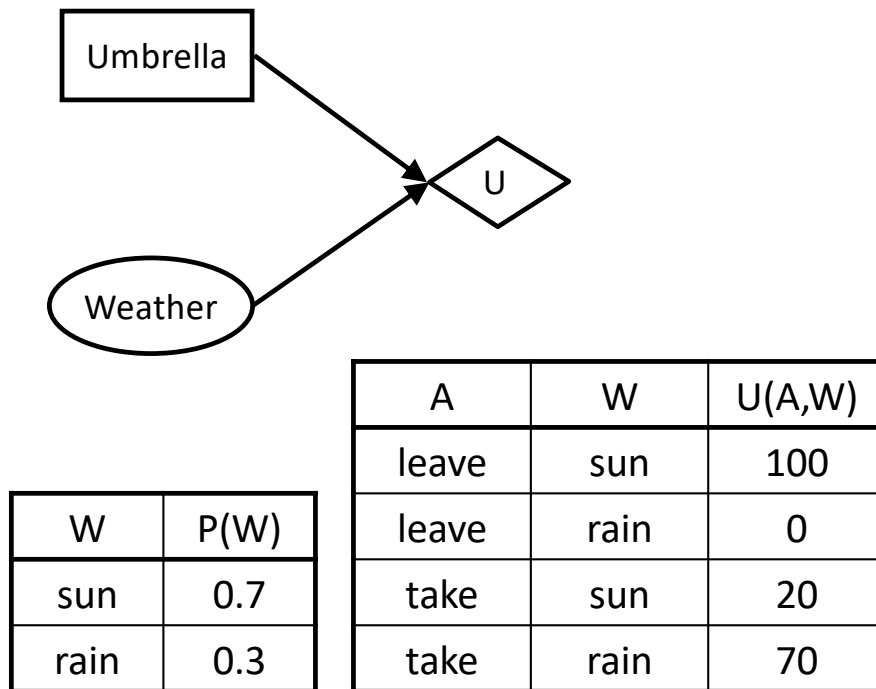
$$\text{EU (action)} = \sum_{\text{outcome}} P(\text{outcome}|\text{action}) \cdot U(\text{outcome})$$

Evaluating Decision Network

- To choose the optimal action, follow these steps:
 - Instantiate evidence variables: Set all known variables for the current state;
 - Enumerate action choices: Assign each possible value to the action node(s);
 - Perform probabilistic inference: For each action, compute the posterior distribution of the utility node's parents, conditioned on evidence;
 - Evaluate expected utility: Use the inferred probabilities to calculate expected utility for each action;
 - Select optimal action: Choose the action that maximizes expected utility.



Example: Evaluating Decision Network



Umbrella = leave

$$\begin{aligned} EU(\text{leave}) &= \sum_w P(w)U(\text{leave}, w) \\ &= 0.7 \cdot 100 + 0.3 \cdot 0 = 70 \end{aligned}$$

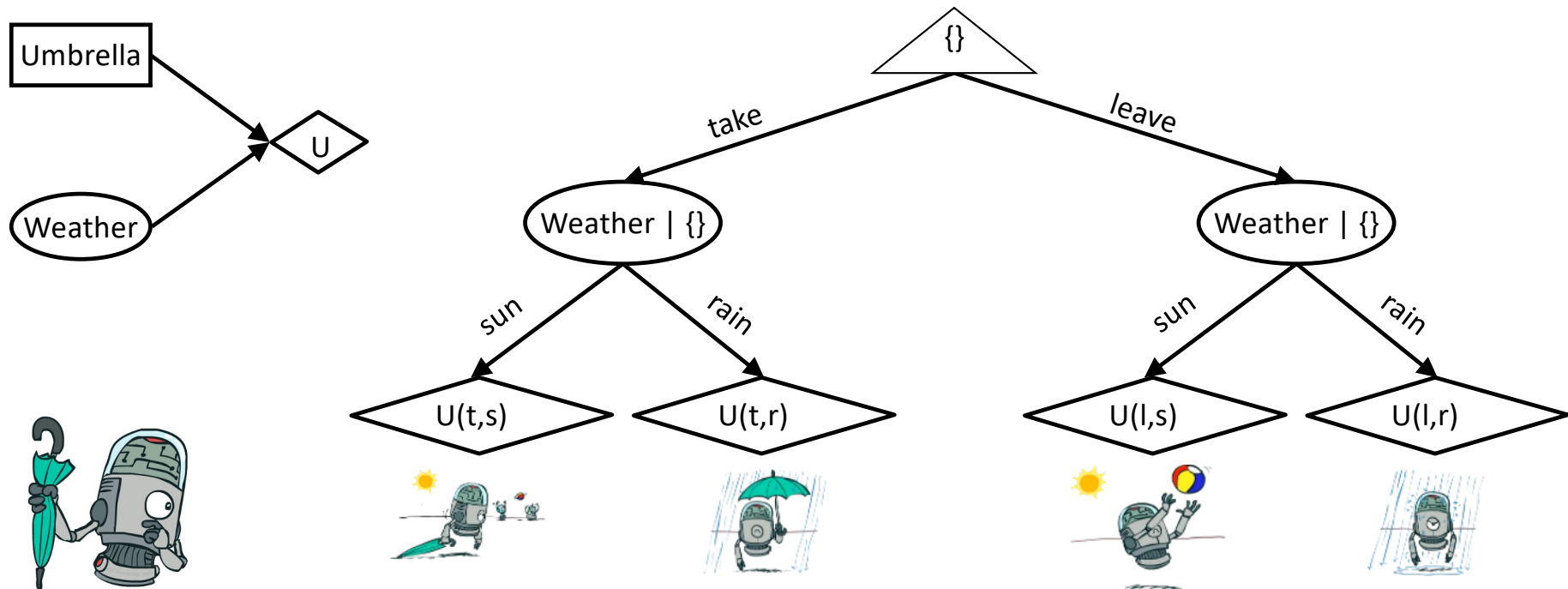
Umbrella = take

$$\begin{aligned} EU(\text{take}) &= \sum_w P(w)U(\text{take}, w) \\ &= 0.7 \cdot 20 + 0.3 \cdot 70 = 35 \end{aligned}$$

Optimal decision = leave

$$MEU(\emptyset) = \max_a EU(a) = 70$$

Decisions as Outcome Trees



MEU Given Evidence

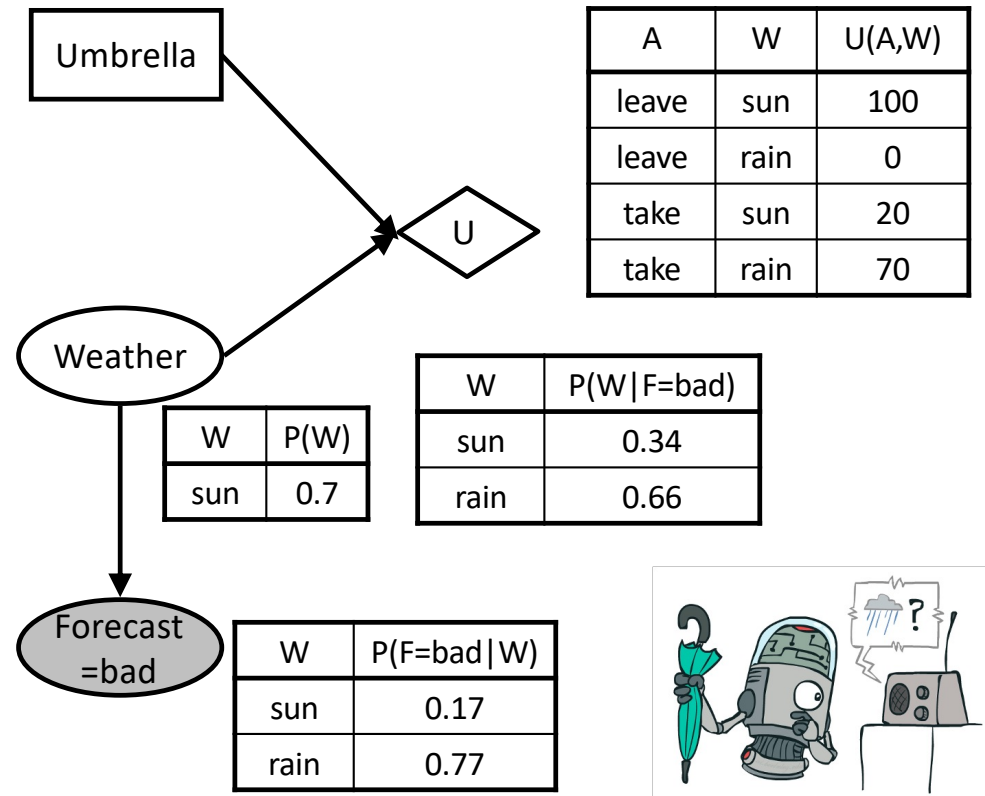
Umbrella = leave

$$EU(\text{leave}|\text{bad}) = \sum_w P(w|\text{bad})U(\text{leave}, w)$$

$$P(W)$$

$$P(W|F) = \frac{P(W, F)}{\sum_w P(w, F)}$$

$$= \frac{P(F|W)P(W)}{\sum_w P(F|w)P(w)}$$



MEU Given Evidence

Umbrella = leave

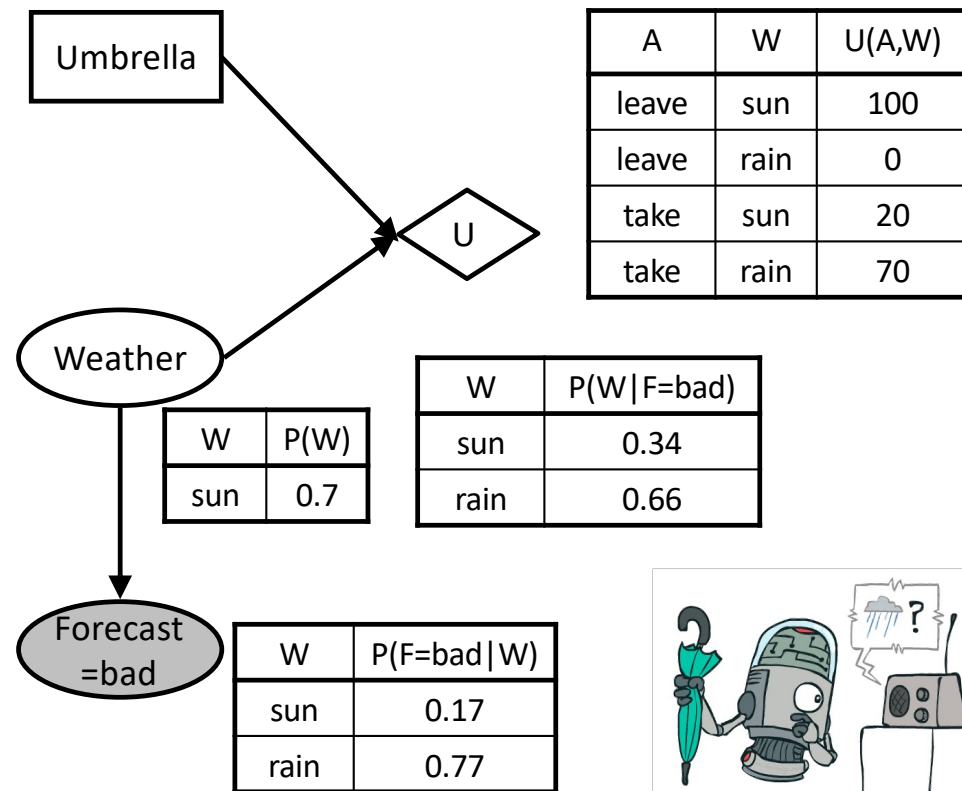
$$\begin{aligned} EU(\text{leave}|\text{bad}) &= \sum_w P(w|\text{bad})U(\text{leave}, w) \\ &= 0.34 \cdot 100 + 0.66 \cdot 0 = 34 \end{aligned}$$

Umbrella = take

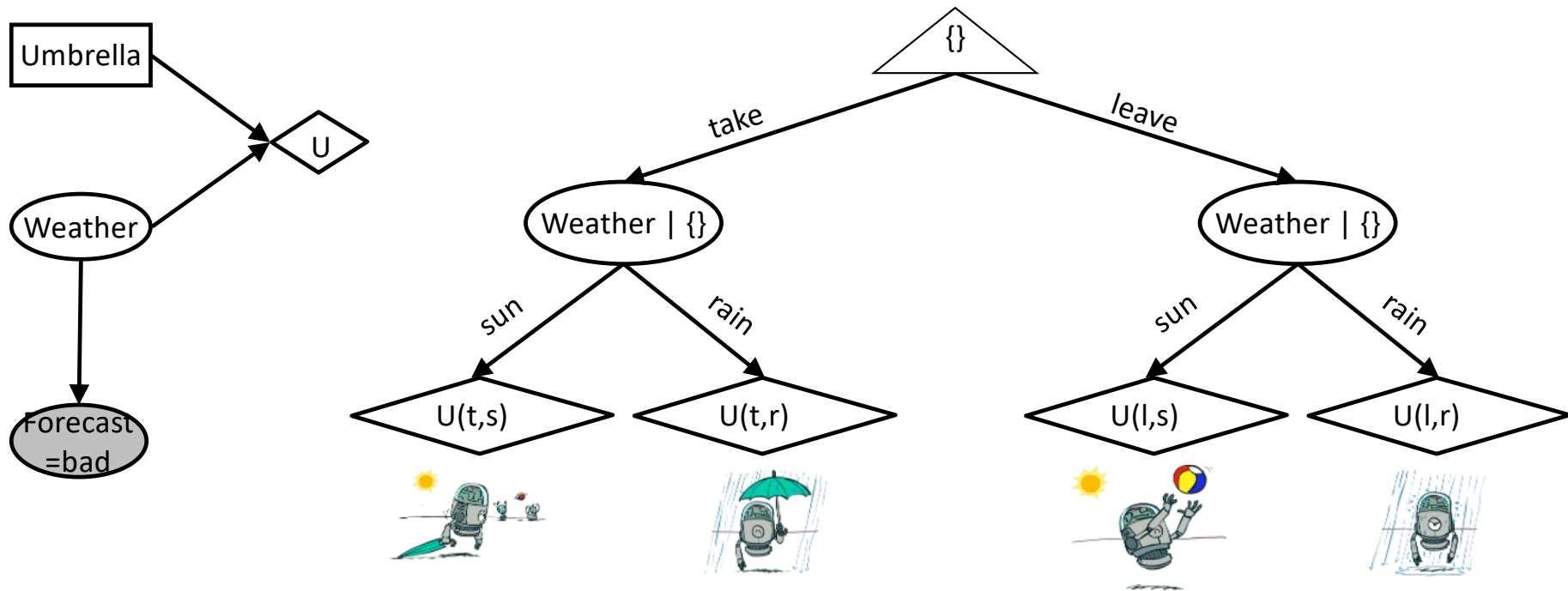
$$\begin{aligned} EU(\text{take}|\text{bad}) &= \sum_w P(w|\text{bad})U(\text{take}, w) \\ &= 0.34 \cdot 20 + 0.66 \cdot 70 = 53 \end{aligned}$$

Optimal decision = take

$$MEU(F = \text{bad}) = \max_a EU(a|\text{bad}) = 53$$

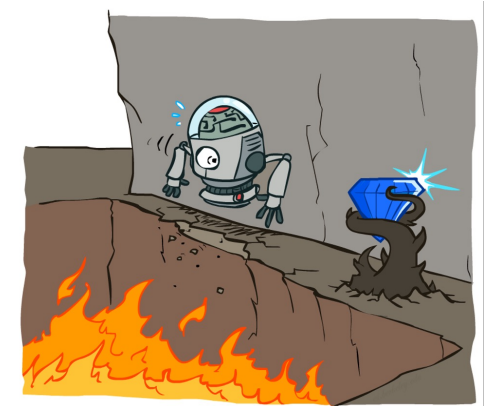


Decisions as Outcome Trees



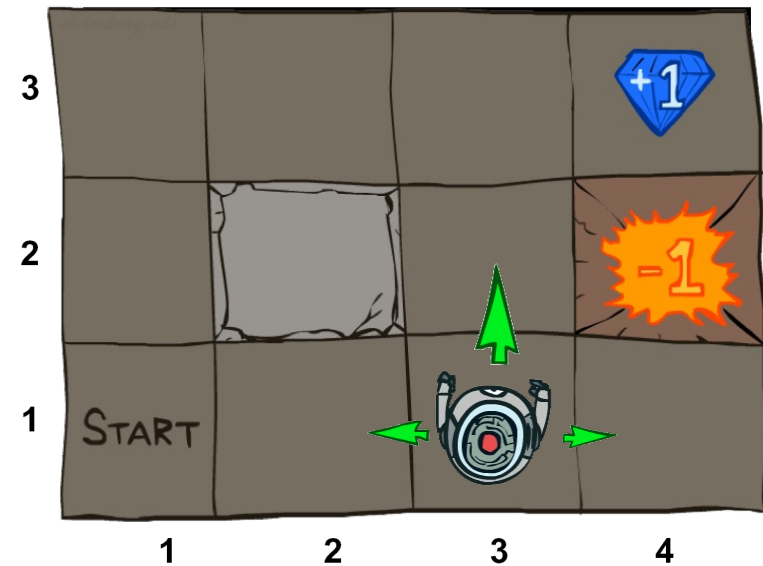
From Episodic to Sequential Decision

- Previous episodic decision network:
 - Focus on single-step decision;
 - The utility of each action's outcome is known and independent;
 - No long-term consequences.
- Sequential decision problem:
 - The agent makes a series of decisions over time;
 - Each decision may influence future states and choices;
 - Utility depends on the entire sequence of decisions;
 - The agent must reason under uncertainty;
 - Requires planning and inference across multiple time steps.



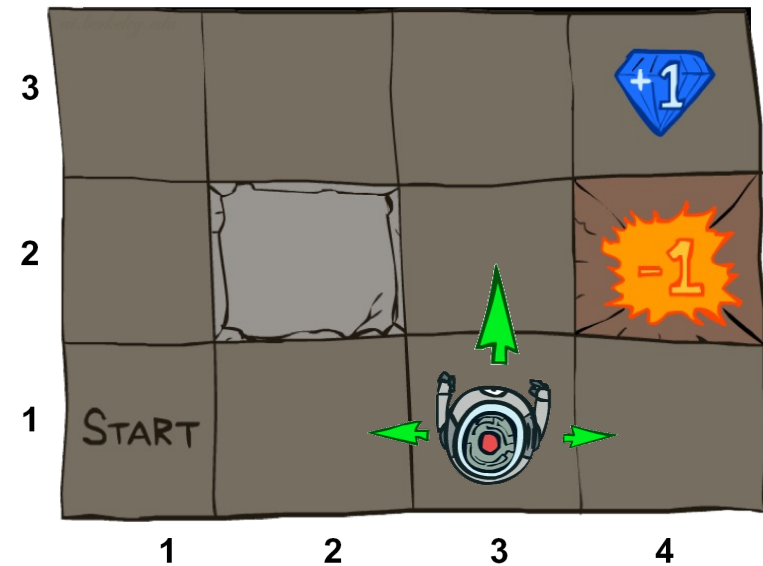
Example: Grid World

- A maze-like problem:
 - The agent operates in a grid world;
 - Walls block certain paths and restrict movement.
- Actions are stochastic: they don't always go as planned.
 - Action "North" moves the agent North with 80% probability (if no wall);
 - Moves West with 10% probability;
 - Moves East with 10% probability;
 - If the intended direction is blocked by a wall, the agent stays in place.



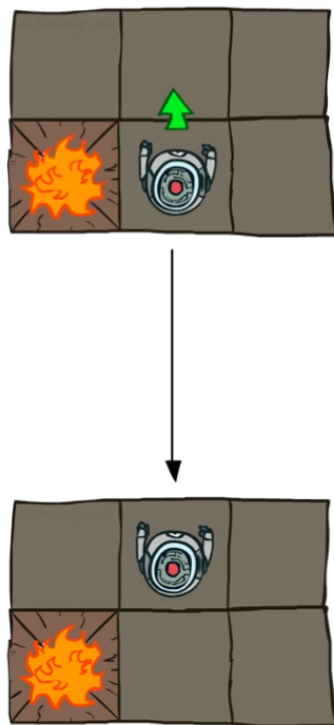
Example: Grid World

- The agent receives rewards:
 - Each step yields a small living reward (can be negative)
 - Terminal states provide large rewards — either positive or negative
- Goal: The agent aims to maximize the total sum of rewards over time.

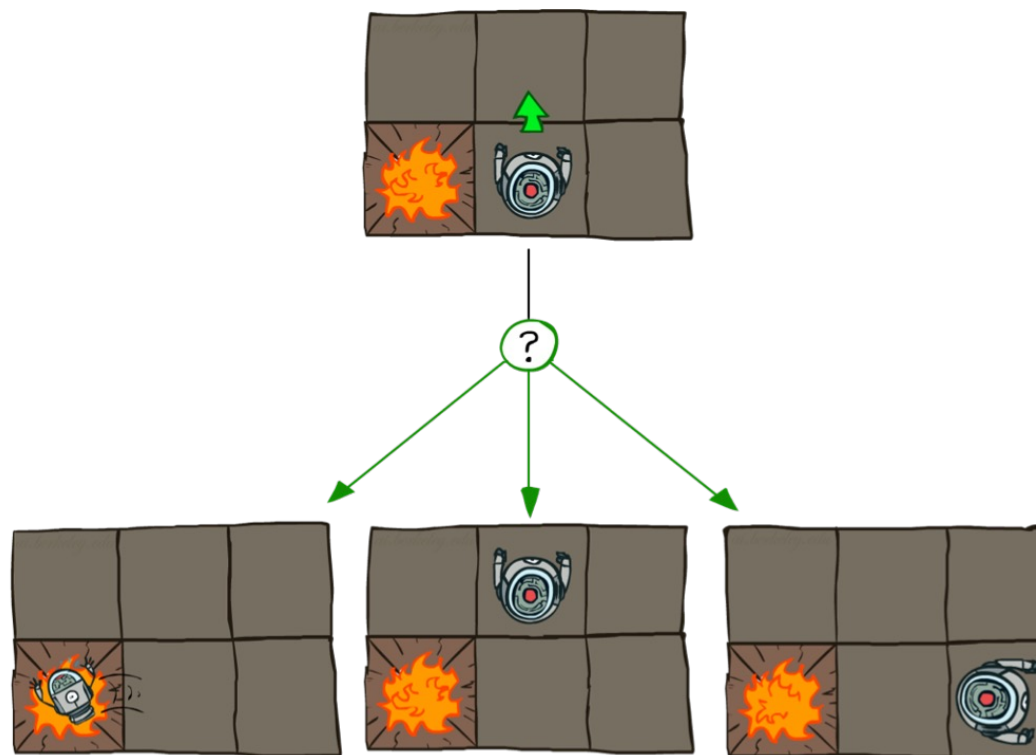


Grid World Actions

Deterministic Grid World

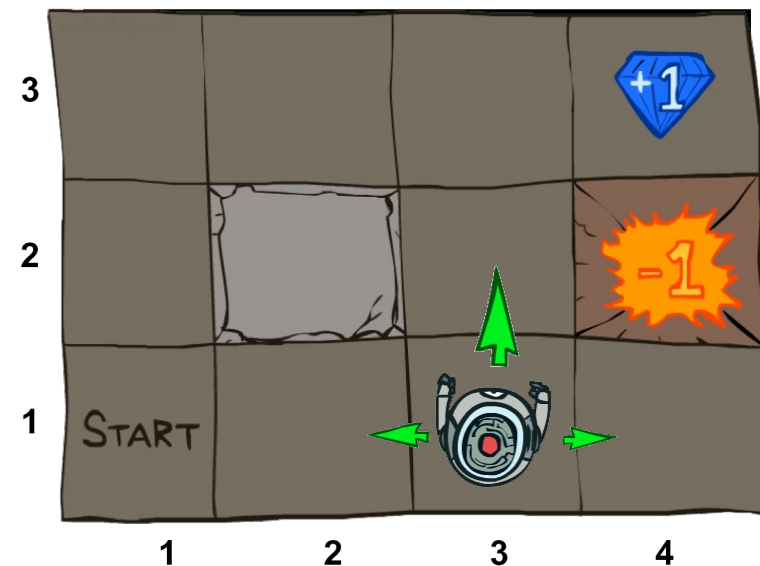


Stochastic Grid World



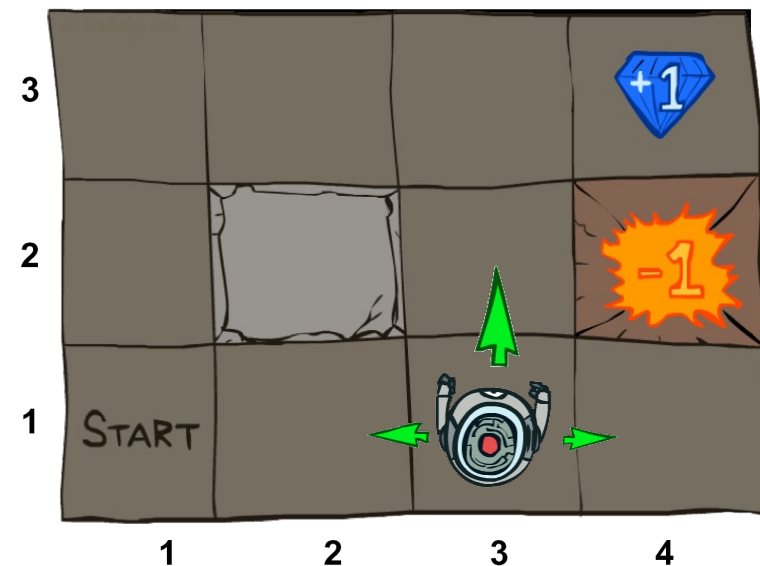
Markov Decision Process (MDP)

- MDP models decision-making under uncertainty, where outcomes are influenced by both randomness and the agent's choices.
 - **States** $s \in S$: All possible situations the agent can be in;
 - **Actions** $a \in A$: Choices available to the agent in each state;
 - **Transition Function** $T(s, a, s')$: Probability of moving from state s to state s' after taking action a (Also called the model or dynamics);



Markov Decision Process (MDP)

- MDP models decision-making under uncertainty, where outcomes are influenced by both randomness and the agent's choices.
 - **Reward Function $R(s, a, s')$** : Numerical feedback for taking action a in state s and ending in state s' (Sometimes simplified as $R(s)$ or $R(s')$);
 - **Start State**: Initial condition of the agent;
 - **Terminal State (optional)**: A state where the process ends.



Markov about MDP

- “Markov” generally means that given the present state, the future and the past are independent.
- For MDP, “Markov” means action outcomes depend only on the current state:

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1}, \dots, S_0 = s_0)$$



$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t)$$



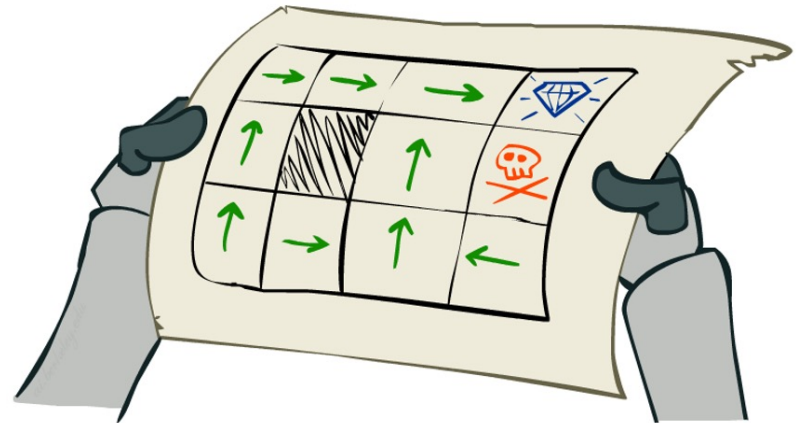
Andrey Markov
(1856-1922)

Policy

- For MDP, our goal is to find an optimal policy:

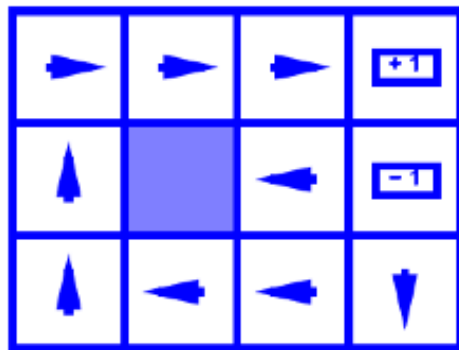
$$\pi^*: S \rightarrow A$$

- A **policy** π maps each state to an action;
- An **optimal policy** π^* maximizes expected utility when followed over time;
- A fully defined policy can be viewed as a **reflex agent**: it specifies exactly what to do in every state.

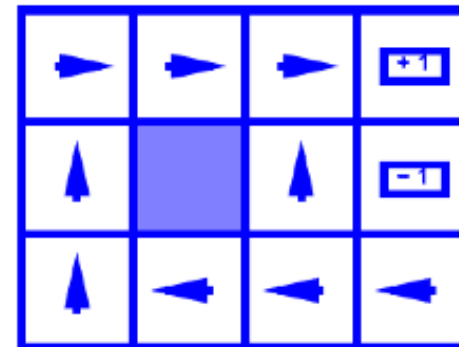


Optimal policy when $R(s, a, s') = -0.03$
for all non-terminals s

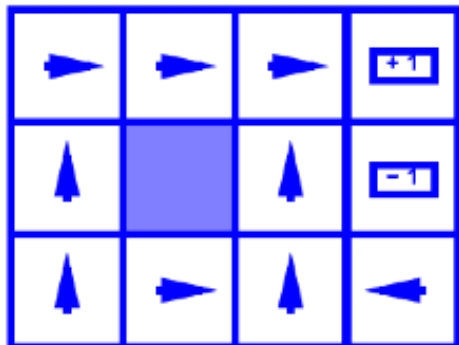
Example: Optimal Policies



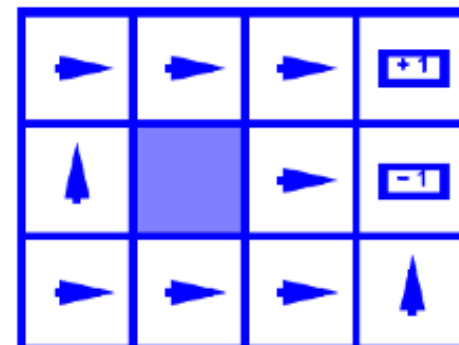
$$R(s) = -0.01$$



$$R(s) = -0.03$$



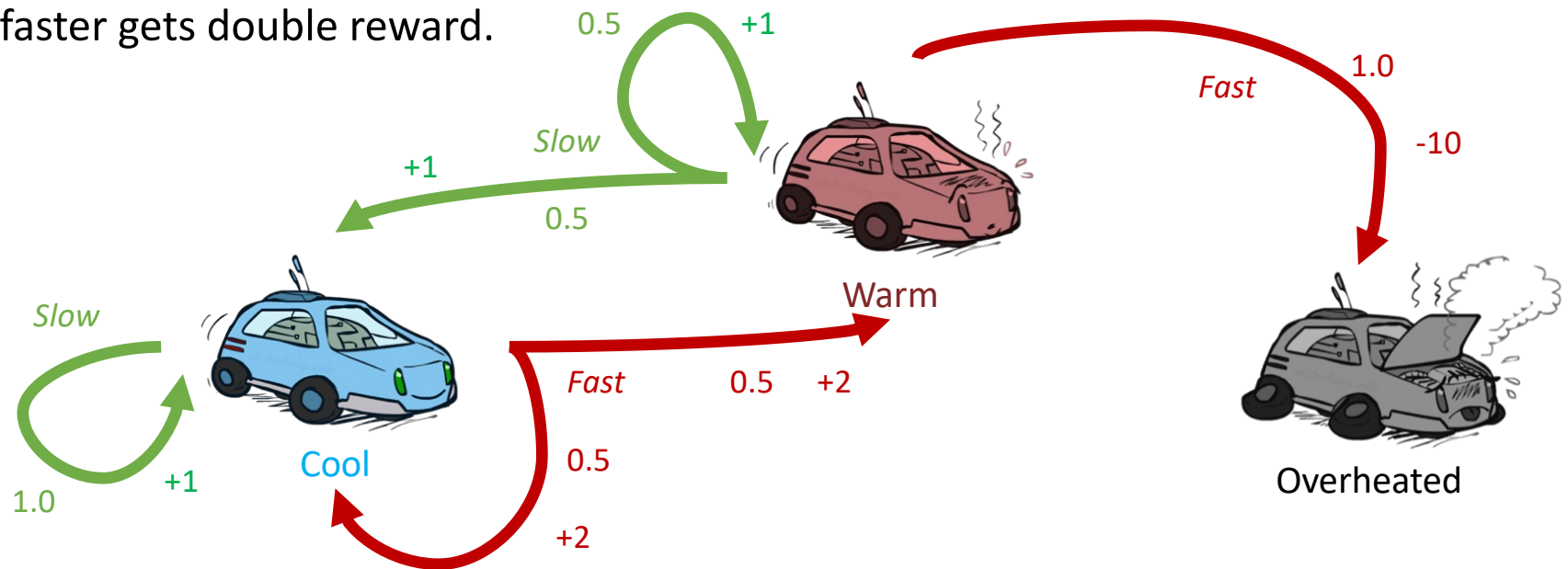
$$R(s) = -0.4$$



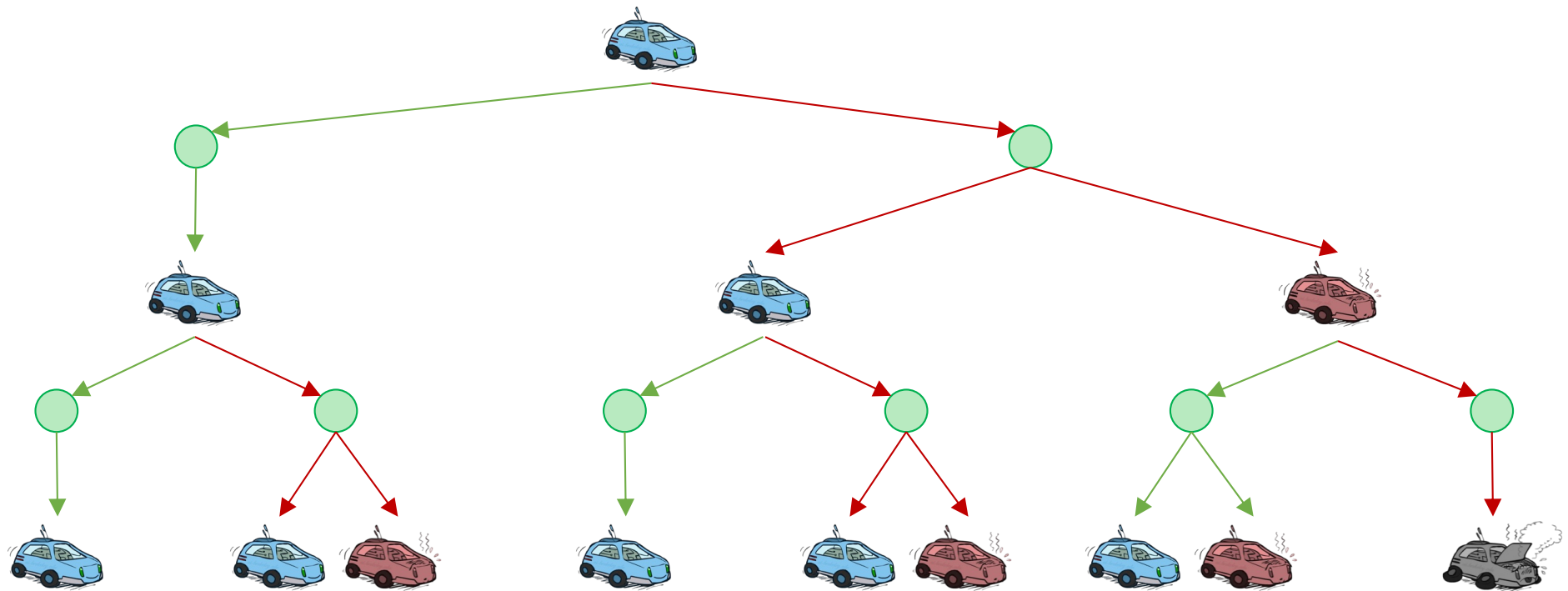
$$R(s) = -2.0$$

Example: Racing

- A robot car wants to travel far, quickly.
- Three states: *Cool*, *Warm*, *Overheated*.
- Two actions: *Slow*, *Fast*.
- Going faster gets double reward.

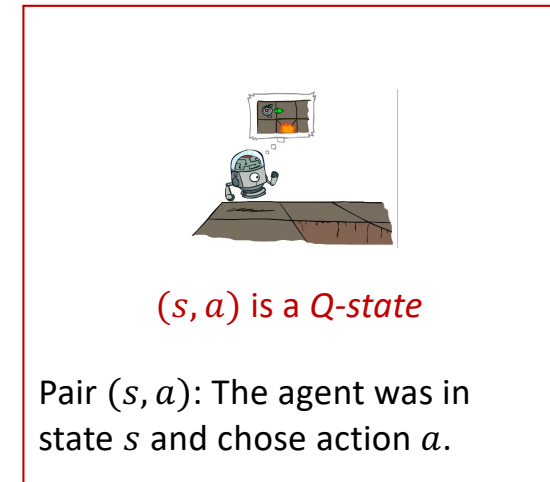
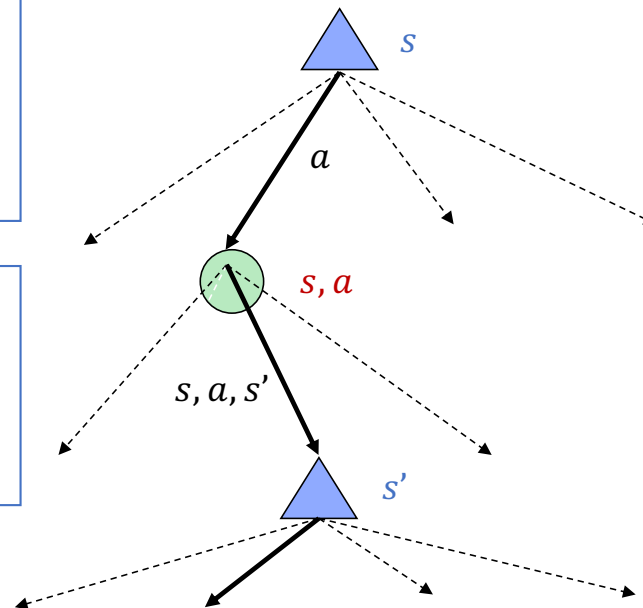
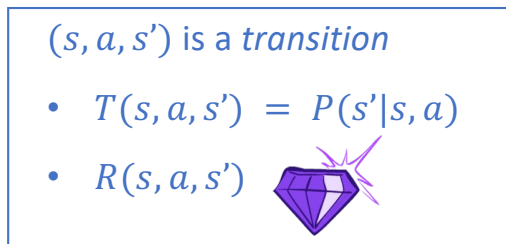
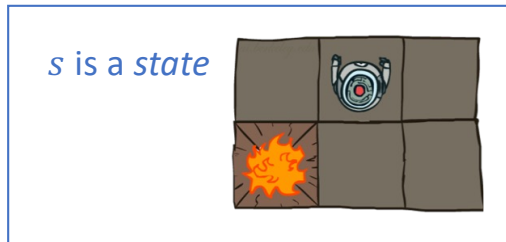


Racing Search Tree

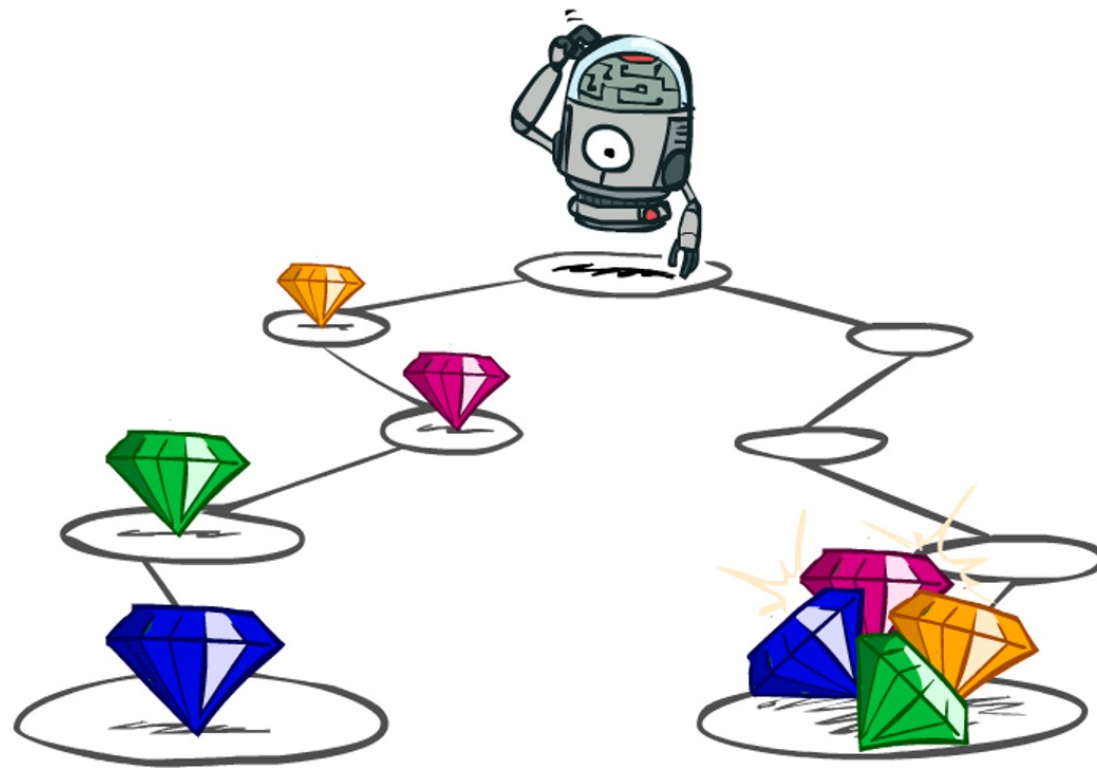


MDP Search Tree

- Each MDP state unfolds into an expectimax-style search tree.
 - Captures both decision choices and probabilistic outcomes.



Utilities of Sequences

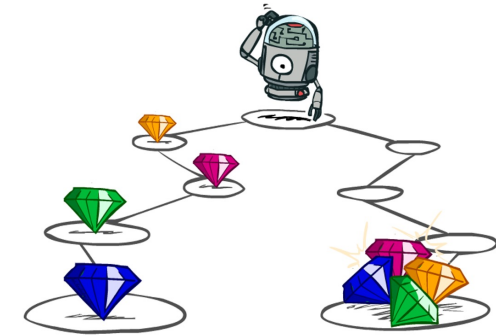


Utilities of Sequences

- What preferences should an agent have over reward sequences?

- More or less? $[1, 2, 2]$ *or* $[2, 3, 4]$

- Now or later? $[0, 0, 1]$ *or* $[1, 0, 0]$



Discounting Utility

- It's reasonable to maximize the total reward an agent receives over time.
- It's also reasonable to prefer rewards sooner rather than later.
 - Factors influencing time preference include: Uncertainty about the future; Risk aversion; Resource constraints; Decay or depreciation of value; Behavioral tendencies (e.g., impulsiveness).
- **Solution: Future rewards are discounted exponentially.**



1

Worth Now



γ

Worth Next Step

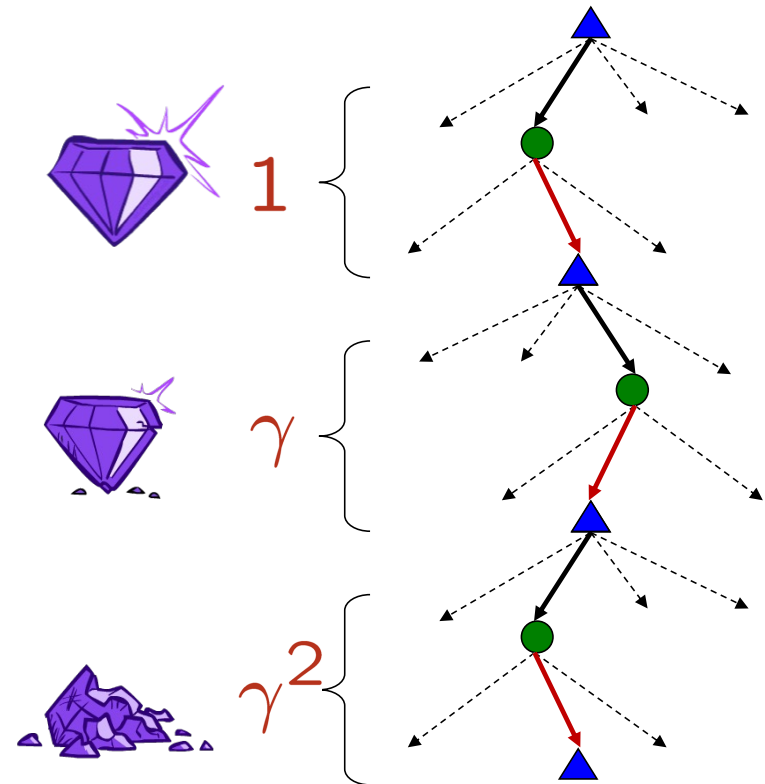


γ^2

Worth in Two Steps

Discounting Utility

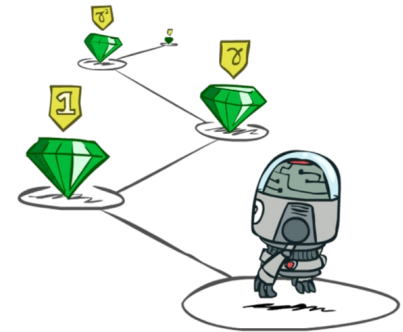
- How to discount?
 - Each time we descend a level in the search tree, we multiply in the discount once.
- Example: discount of $\gamma = 0.5$
 - $U([1,2,3]) = 1*1 + 0.5*2 + 0.25*3$
 - $U([1,2,3]) < U([3,2,1])$



Additive Discounted Reward

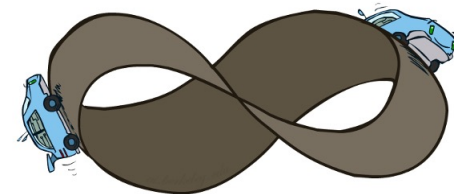
$$U([r_0, r_1, r_2, \dots]) = r_0 + \gamma r_1 + \gamma^2 r_2 \dots$$

- In finite horizon, the agent operates over a limited number of time steps.
 - The total (additive) reward can be explicitly computed.
 - The remaining time may influence decision-making.
- In infinite horizon, the agent has unlimited time to act.
 - The optimal action depends only on the current state, not on time.
 - The resulting optimal policy is stationary (It remains consistent across time).



Convergence of Discounted Reward

- Problem: If the game continues forever, will the agent accumulate infinite rewards?
- Solution 1: Discounting
 - Apply a discounting $0 < \gamma < 1$;
 - Smaller γ implies a shorter effective planning horizon.
- Solution 2: Absorbing state
 - Introduce terminal states that the agent must eventually reach;
 - Guarantees that every policy leads to an end point (like “overheated” state in a racing scenario).



Summary: Defining MDPs

- Markov decision processes:

- Set of states S
- Start state s_0
- Set of actions A
- Transitions $P(s'|s, a)$ (or $T(s, a, s')$)
- Rewards $R(s, a, s')$ (and discount γ)

- MDP quantities so far:

- Policy = Choice of action for each state
- Utility = sum of (discounted) rewards

